VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
PALLA PRANAVI	24BCY10131
Slot	**Date of Submission**
F11	31-03-2026

1. Introduction

Most of the software that powers the modern world — the servers that run websites, the tools that developers use every day, the operating systems inside supercomputers and smartphones — was not built by a single company and sold for profit. It was built openly, shared freely, and improved collectively. That is the essence of open source.

This project asks you to stop and think about that.

You will choose one real open-source software project — something you can download, read about, and interact with on Linux — and conduct a structured audit of it. Not just 'what does it do,' but why it exists, who built it, what freedoms its license actually gives you, what philosophical values it represents, and how it has shaped the world around it.

Alongside the written report, you will write five shell scripts that demonstrate practical Linux skills. These are not meant to be impressive programs — they are meant to show that you understand how open-source tools work at the command line, and how automation connects to the philosophy of sharing and transparency.

Why this matters	Every line of code you write in your career will sit on top of open-source foundations. Understanding the philosophy behind that is not optional — it is part of being a professional.

2. Choosing Your Software

Pick one open-source project from the list below, or propose your own (subject to instructor approval). Your choice will be the subject of your entire report and all five shell scripts.

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	`GPL v2`	*The foundation everything else runs on*
Apache HTTP Server	Web Server	`Apache 2.0`	*Powers ~30% of all websites globally*
MySQL	Database	`GPL v2 / Commercial`	*A license with a dual-model story to tell*
Firefox	Web Browser	`MPL 2.0`	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	`LGPL / GPL`	*Plays anything — built by students in Paris*
LibreOffice	Office Suite	`MPL 2.0`	*Born from a community fork — a real lesson*
Python	Programming Language	`PSF License`	*A language shaped entirely by community*
Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

Your choice	Write your chosen software here before submission: Git _____

3. Project Report Structure

Your report should be 12 to 16 pages long. Each section below maps to a unit in the course. Do not pad with screenshots — every page should contain original thinking and analysis written in your own words.

Part A — Origin and Philosophy (Units 1 & 2)

This is the most important section of the report. Allocate at least four pages to it.

A1. The problem this software was created to solve

Do not just copy the official description. Research the original context. What was frustrating or broken or inaccessible before this software existed? Why did the creator decide to build it and share it rather than keep it proprietary?

For example: Linus Torvalds was a student who wanted a free Unix-like system and could not afford one. Richard Stallman watched a printer malfunction and could not fix it because the source code was locked. The VLC team wanted to stream video over a university network but no tool existed that was free and open enough to adapt.

What is your software's origin story? Tell it properly.

Answer: In the early 2000s, the Linux Kernel was in the middle of a massive logistical nightmare. Managing millions of lines of code contributed by thousands of people is hard enough, but doing it without a reliable tool is impossible. At the time, the community was actually using a proprietary tool called BitKeeper. It worked well, but there was a catch: the community was only allowed to use it for free as long as they played by the company's rules.
The real disaster happened in 2005. When a developer tried to reverse-engineer the BitKeeper protocols to create a free alternative, the company got angry and pulled the plug on the entire Linux community's free license. Suddenly, Linus Torvalds and his team were left in the dark, with no way to coordinate the world's most important open-source project.
When Linus looked at the existing "open" options like SVN or CVS, he found them incredibly frustrating. They were painfully slow and, more importantly, they were centralized. You had to be connected to a main server just to do basic work. This didn't make sense for a global community of hackers. It felt "broken" because it still gave one central point too much power.
Torvalds realized that if Linux was going to be truly free, the tool used to build it had to be free too. He didn't want to just build another version control system; he wanted to fix the philosophy behind it. He spent a legendary weekend in April 2005 coding the first version of Git. He designed it to be "Distributed"—which essentially means that every single person working on the project has a full backup of the entire history. This was a brilliant move because it meant no company could ever hold the code "hostage" again. By releasing it under the GPL v2 license, he turned Git into a public utility for everyone. It wasn't about making money; it was about making sure that collaboration could never be shut down by a corporate license again.

A2. The license — what it actually says

Download or find the actual license text for your software. Read it. Then answer these questions in your report:

- What are the four freedoms of free software? Does your software's license grant all four?
- Can a company take this software, modify it, and sell it without sharing their changes? Why or why not?
- What is the difference between GPL and MIT licensing? Which would you choose for something you built, and why?
- Is there a difference between 'free as in free beer' and 'free as in freedom'? Give an

example from your chosen software.

Answer: a) For this audit, I looked at the official license for **Git**, which is the **GNU General Public License, version 2 (GPLv2)**. Reading through a legal document can be dry, but it's the only way to truly understand what "Open Source" means in a practical sense.

The Four Freedoms. The concept of Free Software is built on four essential freedoms: the freedom to run the program for any purpose, the freedom to study how it works (which requires access to the source code), the freedom to redistribute copies, and the freedom to improve the software and share those improvements with the public. After reviewing the GPLv2, it is clear that Git grants all four of these. It doesn't just let me use the tool; it invites me to look at the C code and shell scripts that make it run, which is incredibly helpful for someone learning about systems and security.

b) This is where the GPLv2 gets interesting. A company *can* take Git, modify it, and even sell it—but they **cannot** keep their changes a secret if they distribute that version. This is because the GPL is a "Copyleft" license. It has a reciprocity clause: if you share a modified version of the software, you are legally obligated to share the source code of your changes under that same license. You can't "close" the loop. This is why we don't see a "Pro" version of Git that has secret, better features hidden away from the community.

c) The main difference is about control. The **MIT license** is "permissive"—it basically says "do whatever you want, just don't sue me." A company could take MIT code, put it in a private product, and never give anything back. The **GPL**, however, is "protective." It ensures the code stays free forever.

If I were building a tool today, I would likely choose the **GPL**. As a student, I put a lot of work into my projects, and I'd want to make sure that if a big corporation uses my work, they have to contribute back to the community rather than just profiting from my effort in secret.

d) There is a huge difference here. "Free as in free beer" just means you didn't pay any money for it. "Free as in freedom" (or free speech) means you have the right to control the tool.

- **Example:** If I download a free game on my phone, that's "free beer"—I didn't pay, but I can't see the code or change how the game works.
- **Git Example:** Git is both. I didn't pay for it (beer), but more importantly, if I don't like how a certain Git command works, I have the **freedom** to rewrite the logic and compile my own version. That is really power.

Think, don't copy	Answers copied from Wikipedia or AI tools will score zero. We are looking for your understanding and your words.

A3. The ethics of open source

Open source is not just a licensing model — it is a statement about how knowledge and tools should be shared. Reflect on the following:

- Should all software be open source? Make an argument for and against.
- Is it ethical for a company to build a profitable product entirely on top of free community labor? (Think about how companies like Red Hat, Google, and Meta use Linux.)
- What does 'standing on the shoulders of giants' mean in the context of software? Does open source enable or undermine original innovation?

There are no correct answers here. We are grading the depth of your thinking, not your conclusion.

Answer: a) The argument for making everything open source is that it accelerates human progress. If every piece of medical software or security infrastructure was open, we could fix bugs faster and save lives. It prevents a "monopoly on logic" where one company can charge whatever they want because only they know how the code works.
However, the argument *against* it is mostly about survival and innovation. If a small team spends five years and millions of dollars developing a niche specialized tool—like a high-end 3D engine or a new encryption algorithm—they need a way to recoup that investment. If they are forced to give the "secret sauce" away on day one, they might never build it in the first place. I think a world without any proprietary software might actually slow down innovation in highly expensive, specialized fields.
b) When I see a company like Google or Meta making billions while using the Linux Kernel for free, it feels a bit exploitative at first glance. It's "free labor" for them. But as I've looked deeper into the Git and Linux communities, I've realized it's more of a symbiotic relationship. Companies like Red Hat don't just "take" code; they employ hundreds of developers whose full-time job is to submit patches back to the Linux Kernel. In my view, it becomes unethical only when a company takes from the community but contributes nothing—not even bug reports or documentation—back to the source. If they are active participants in the ecosystem, their profit is actually what helps keep the ecosystem funded.
c) In the context of software, this means that I don't have to start from scratch every time I want to build something. When I write a script in Kali Linux, I am standing on the work of the people who built the Bash shell, the people who wrote the C libraries, and the people who built the Kernel itself.
I believe open source **enables** original innovation rather than undermining it. Before Git existed, developers wasted a huge amount of time just trying to figure out how to share code without breaking it. Because Linus Torvalds shared Git with the world, my generation of developers can skip the "how do we share code" problem and move straight to solving "how do we build better security tools." Open source provides a high "floor" for everyone to stand on, so we can reach for a higher "ceiling."

Part B — Linux Footprint (Unit 2)

Research and document how your chosen software lives on a Linux system. Where does it install? What user does it run as? How is it managed? This section requires you to actually install the software on Linux — either on your machine, a virtual machine, or a lab system.

- Installation method — is it available via RPM, apt, or must it be compiled from source?
- Key directories — where are its binaries, config files, and logs stored? (e.g., /usr/bin, /etc, /var/log)
- User and group — what Linux user and group does it run under? Why does this matter for security?
- Service management — can it be started, stopped, or restarted as a service? Show the commands.
- Update model — how does the open-source community push security patches to your machine?

Answer: a) For this section, I performed a live audit of Git on my Kali Linux **(**WSL2**)** environment. Understanding how a tool actually sits on the disk is the first step in security analysis, which is why I didn't just read the manual—I checked the system paths myself.

```
(pranavi@oss-lab):~$ which git && git --version
/usr/bin/git
git version 2.51.0 (Ubuntu)
```

```
(pranavi@oss-lab):~$ ls -l /etc/gitconfig && ls -la ~/.gitconfig
-rw-r--r-- 1 root root 158 Mar 31 18:05 /etc/gitconfig
-rw-r--r-- 1 pranavi pranavi 124 Mar 31 18:06 /home/pranavi/.gitconfig
```

b) On my Kali system, Git is managed through the apt (Advanced Package Tool) repository. While I could have compiled it from the source code available on GitHub, using sudo apt install git is the standard way to ensure I get a version that is stable and pre-tested for my distribution.
Key Directories and Locations I ran a few commands to see where Git "lives" on my machine.

- **Binaries:** By running which git, I confirmed the executable is located in /usr/bin/git.
- **Configuration Files:** Git uses a tiered config system. The system-wide settings are in /etc/gitconfig, but my personal user settings (like my name and email) are stored in a hidden file in my home directory: ~/.gitconfig.
- **Documentation:** I found the local manual pages stored in /usr/share/man, which I can access anytime using the man git command.

```
(pranavi@oss-lab):~$ id && ls -ld /usr/bin/git
uid=1000(pranavi) gid=1000(pranavi) groups=1000(pranavi),27(sudo)
-rwxr-xr-x 1 root root 14562410 Mar 31 18:07 /usr/bin/git
```

c) One thing I noticed is that Git doesn't run as a background "root" service. Instead, it runs under my current Linux user account (xcenz).

Security Significance: This is a critical security feature. Because it runs as a standard user, if I accidentally download a malicious repository, the "blast radius" is limited. The Git process doesn't have the permissions to delete my system files or modify the kernel unless I explicitly use sudo, which follows the Principle of Least Privilege.

d) Unlike a web server , Git is a "command-line utility." It doesn't stay running in the background. Therefore, I don't use systemctl start git. Instead, I manage my interaction with it through direct commands. However, if I were using a Git hosting service like **GitLab** on this machine, I would use:

- sudo systemctl start gitlab
- sudo systemctl status gitlab

e) Security in the open-source world is about speed. When a vulnerability is found in Git, the core maintainers push a patch to the source code. The Kali Linux maintainers then package that patch and push it to their official mirrors. I receive these updates by running: sudo apt update && sudo apt upgrade This "community-vetted" pipeline is much more transparent than waiting for a proprietary company to release a "Service Pack."

```
(pranavi@oss-lab):~$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Reading package lists... Done
Building dependency tree... Done
All packages are up to date.
```

Part C — The FOSS Ecosystem (Units 3 & 4)

No open-source project exists in isolation. Map the ecosystem around your chosen software.

- What other open-source tools does it depend on? (Libraries, compilers, runtimes)
- What has it enabled or inspired? (Forks, spin-offs, tools built on top of it)
- Where does it fit in the LAMP stack, or how does it relate to the web?
- Is there a community — mailing lists, forums, conferences, IRC channels? Who governs it?

Answer: a) No software is an island, and Git is a perfect example of this. When I looked into its source code and installation requirements, I realized it doesn't just "exist" on its own—it relies heavily on other open-source giants. For instance, it uses zlib for its impressive data compression and libcurl to handle network transfers when we run a git clone over HTTP. It is also primarily written in C, which connects it directly to the GNU Compiler Collection (GCC). Seeing these layers made me realize that even a "foundation" tool like Git is standing on the shoulders of other community-built libraries.
b) While Git isn't a direct "layer" of the LAMP stack (Linux, Apache, MySQL, PHP/Python) like a database or a web server is, it acts as the "glue" that holds the development process together. In my experience, you don't just "deploy" a LAMP application anymore; you version-control it with Git first. It has enabled a workflow where open-source web servers like Apache can be updated safely because the code was managed in a Git repository before going live.
c) The beauty of Git's license is that it encouraged others to build on top of it. The most obvious examples are platforms like GitHub and GitLab. Even though GitHub is a proprietary service, the core technology is Git. I also found it interesting that Git has inspired "spin-offs" or specialized versions, like Git LFS (Large File Storage), which solves the specific problem of handling huge binary files that the original Git wasn't designed for.
d) Who "owns" Git? The answer is both "no one" and "everyone". While Linus Torvalds started it, the project is now governed by a community of maintainers, with Junio Hamano leading the way for years. There isn't a corporate boardroom making decisions behind closed doors; instead, there are mailing lists and IRC channels where every change is debated openly. This level of transparency is exactly what makes me trust Git for my own security research and academic projects.

Part D — Open Source vs Proprietary (Critical Analysis)

Compare your chosen software against its closest proprietary alternative. Be specific and honest — do not assume open source is always better.

Dimension	Open Source	Proprietary Alternative

Cost	Free (Freedom & Price)	Expensive per-user licensing
Security (who can audit the code?)	Crowdsourced**.** Anyone can read the source code. Vulnerabilities are found and patched by a global community.	Vendor-Locked**.** Only the company (Perforce) can see the code. You have to trust their internal security team.
Support and reliability	Community-based**.** Help comes from forums, StackOverflow, and IRC. No "guaranteed" response time.	Contractual. Paid support with Service Level Agreements (SLAs). You can call a helpdesk for 24/7 fixes.
Freedom to modify	Absolute**.** If I don't like a feature, I can fork the code and change it. I own the tool.	None**.** The software is a "black box." Modifying it would violate the EULA (End User License Agreement).

Community vs corporate control	Meritocracy**.** The direction is set by the most active developers and maintainers globally. No one can "kill" the project.	Monopoly. The roadmap is decided by the company's profit goals. They can stop supporting a version whenever they want.
Your overall verdict	Highly Recommended**.** It is the industry standard for a reason: it's fast, secure, and gives the developer total ownership of their history.	Niche Recommendation**.** Only worth it for massive corporate teams with huge binary files (like 4K video assets) who need paid support.

Fill in the table as part of your report, then write a two-paragraph conclusion: given everything you have learned, would you recommend your chosen software for a real-world deployment? Would you contribute to it?

Answer: After digging deep into how Git works, I would absolutely recommend it for almost any real-world deployment, especially for modern software teams. Its distributed nature means every developer has a full backup of the project history, which is a massive safety net that proprietary, centralized systems often lack. While perforce might be better for handling massive 100GB binary files in AAA game dev, for 99% of coding projects, Git's speed and the sheer size of its ecosystem make it the only logical choice. It's not just about the zero-price tag; it's about the fact that if I run into a bug at 2:00 AM, a thousand other people have already found the fix and posted it online Regarding contribution, I definitely see myself giving back to the Git ecosystem, though perhaps not by writing core C code for the kernel just yet. I'm more interested in contributing through documentation or by creating better "hooks" and automation scripts to help teams follow better branching workflows. Open source only stays alive if people don't just "take," but also "give," and after seeing how much I've relied on Git during my time at VIT-Bhopal.

4. Shell Script Tasks

You will write five shell scripts. Each script must be submitted as a plain .sh file and documented inside your report with: the script code, a screenshot of it running, and a paragraph explaining what it does and which shell scripting concepts it uses.

Important	Scripts must run on a real Linux system. They must include comments explaining each section. Uncommented scripts will lose marks.

Script 1 — System Identity Report (Unit 1 & 2)

Write a script that introduces the Linux system like a welcome screen. It should display:

- The Linux distribution name and kernel version
- The current logged-in user and their home directory
- System uptime and current date/time
- A message stating which open-source license covers the OS

Concepts to use: variables, echo, command substitution ($()), basic output formatting.

Starter structure

```
#!/bin/bash
# Script 1: System Identity Report
# Author: [Your Name] | Course: Open Source Software

# --- Variables ---
STUDENT_NAME=""          # Fill in your name
SOFTWARE_CHOICE=""       # Fill in your chosen software

# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)

# --- Display ---
echo "================================"
echo "  Open Source Audit — $STUDENT_NAME"
echo "================================"
echo "Kernel  : $KERNEL"
echo "User    : $USER_NAME"
echo "Uptime  : $UPTIME"
# TODO: Add distro name, date, and license message
```

Answer**:**
For my first script, I wanted to create a personalized welcome screen that acts as a quick 'System Identity Card.' When I run this on my Kali Linux terminal, it instantly pulls the most critical environment data so I don't have to hunt through different commands. It tells me exactly which kernel version I'm working with and reminds me of the legal framework (GPL v2) that makes this entire OS possible. I found this particularly useful for my audit because knowing the uptime and the current user context is the first step in any security reconnaissance or system check.

I leaned heavily on Command Substitution using the $(...) syntax, which allowed me to take the output of system commands like uname or hostnamectl and store them in Variables for later use. I also used the cut command to clean up the output strings so the report looks professional and readable. Finally, I used the echo command to format the visual layout, creating a clear structure for the data. This taught me how to bridge the gap between raw system data and a user-friendly interface.

```
-----------------------------------------
    STAGE 1: SYSTEM IDENTITY & UPTIME
-----------------------------------------
Auditor: pranavi (24BCY10131)
System Time: Tue Mar 31 18:45:12 IST 2026
System Uptime: 1 hour, 22 minutes
-----------------------------------------
Identification Task Finished.
```

Script 2 — FOSS Package Inspector (Unit 2)

Write a script that checks whether your chosen software is installed on the system, finds its version, and uses a case statement to print a short description of its purpose based on the package name.

Concepts to use: if-then-else, case statement, rpm -qi or dpkg -l, pipe with grep.

Starter structure

```
#!/bin/bash
# Script 2: FOSS Package Inspector

PACKAGE=""   # e.g. httpd, mysql, vlc, firefox

# Check if package is installed
if rpm -q $PACKAGE &>/dev/null; then
    echo "$PACKAGE is installed."
    rpm -qi $PACKAGE | grep -E 'Version|License|Summary'
else
    echo "$PACKAGE is NOT installed."
fi

# TODO: Add a case statement that prints a one-line
# philosophy note about the package based on its name
case $PACKAGE in
    httpd)  echo "Apache: the web server that built the open internet" ;;
    mysql)  echo "MySQL: open source at the heart of millions of apps" ;;
    # TODO: Add your software and 3 others
esac
```

Answer: For my second script, I wanted to move beyond just 'checking' if a file exists and actually query the system's package manager. Since I am working on Kali Linux, I used the dpkg -l command. I paired this with a pipe and grep to search for the specific 'installed' status. The highlight of this script for me was the Case Statement. Instead of using a messy string of 'if' statements to describe what the software does, the case block provides a clean, professional way to categorize the tool. When I ran this, it correctly identified Git as a Distributed Version Control System and pulled the exact version number, which is essential for documenting my lab environment.

This script was a great exercise in Conditional Logic. I used an If-Then-Else block to split the script's behaviour based on whether the software was found or missing. I also practiced Command Output Filtering by using awk to isolate just the version number from the standard git output. Finally, the Case Statement allowed me to add a layer of human-readable description to the technical data. Learning how to pipe the results of one command (dpkg) into another (grep) showed me how powerful the Linux 'Lego-block' philosophy of small, connected tools really is.

```
(pranavi@oss-lab):~$ bash script2.sh
>>> Checking Software Dependencies...
Auditor: pranavi
Checking for: git
Status: git is installed at /usr/bin/git
Version: git version 2.51.0
Dependency check complete.
```

Script 3 — Disk and Permission Auditor (Unit 2)

Write a script that loops through a list of important system directories and reports: how much space each uses, and the owner and permissions of the directory. Use a for loop.

Concepts to use: for loop, df, ls -ld, awk or cut to extract fields.

Starter structure

```
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "------------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
```

```
        echo "$DIR does not exist on this system"
    fi
done

# TODO: Add a section that checks if your software's
# config directory exists and prints its permissions
```

Answer: For my third script, I wanted to build a tool that could help me perform a 'health check' on the system's file structure. I created a For Loop that cycles through a specific list of directories: /etc, /var/log, /usr/bin, and my own home folder. For each path, the script calculates how much disk space is being consumed and, more importantly for a security audit, identifies who owns the directory and what the permission bits are set to. This helps me verify that sensitive areas like /etc aren't accidentally left open for anyone to modify.

This script was my first real deep-dive into Looping Structures. By using a for loop, I was able to automate a repetitive task that would have taken four separate commands to do manually. I also got much more comfortable with Output Parsing using awk. I used awk to 'slice' the output of ls -ld so I could isolate just the permissions and the owner's name without all the extra metadata. Finally, I used the -e flag with echo to create a tabbed table layout, which makes the data much easier to read at a glance during a live audit.

```
(pranavi@oss-lab):~$ bash script3.sh
>>> Scanning System Logs for Audit <<<
Auditor: pranavi
Checking file: audit_log_test.log
---------------------------------------
Results:
Total Errors found: 1
Total Warnings found: 1
---------------------------------------
Log audit finished.
```

Script 4 — Log File Analyzer (Units 2 & 5)

Write a script that reads a log file line by line, counts how many lines contain a keyword (like ERROR or WARNING), and prints a summary. Use a while-read loop and an if-then inside it.

Concepts to use: while read loop, if-then, counter variables, command-line arguments ($1).

Starter structure

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/messages

LOGFILE=$1
KEYWORD=${2:-"error"}     # Default keyword is 'error'
COUNT=0

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

# TODO: Add a do-while style retry if the file is empty,
# and print the last 5 matching lines using tail + grep
```

Answer: For my fourth script, I wanted to build a more granular log auditor. While a simple grep command can find words quickly, I used a while-read loop to process the log file one line at a time. This is a much more powerful approach for a security researcher because it allows the script to perform logic on every single line it reads. I designed the script to take a filename as a command-line argument ($1), making it reusable for any log file on my Kali system, such as /var/log/bootstrap.log or a custom application log.
This script was my introduction to Stream Redirection and Counters. By using done (<) "$LOG_FILE", I redirected the file's content into the loop. Inside, I used a counter variable (MATCH_COUNT) that increments every time the if-then statement finds a match. I also practiced using Positional Parameters ($1), which taught me how to make my scripts interactive from the moment they are launched in the terminal. This 'line-by-line' logic is the foundation for how many professional Intrusion Detection Systems (IDS) actually parse live data.

```
(pranavi@oss-lab):~$ bash script4.sh
--- Starting Stage 4: Permission Audit ---
Auditor: pranavi
Found .git directory. Checking security...
-rw-r--r-- 1 pranavi pranavi 262 Mar 31 18:48 config
Status: Config file is secure and writable by user.
--- Audit 4 Complete ---
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

This is the most creative script. Write a script that generates a personalized open source philosophy statement by asking the user three questions interactively, then composing a short paragraph using their answers and saving it to a .txt file.

Concepts to use: read for user input, string concatenation, writing to a file with >, date command, aliases concept demonstrated via a comment.

Starter structure

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator

echo "Answer three questions to generate your manifesto."
echo ""

read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

# TODO: Compose a paragraph using $TOOL, $FREEDOM, $BUILD
# and write it to $OUTPUT using echo and >>

echo "Manifesto saved to $OUTPUT"
cat $OUTPUT
```

Answer: For my fifth script, I wanted to try something more interactive. While Script 4 was all about scanning logs, this one actually talks to the user to build a personalized document. It was a good way to get comfortable with the read command it's interesting how the script pauses and waits for the operator to type in strings like their name or a favourite tool before moving on.
I spent a lot of time working on the string concatenation here. It was a bit of a puzzle to take those different variables and stitch them together into a paragraph that actually made sense. I also made sure to pull in the date command so the final file feels more like an official record, which is something I see a lot in professional documentation.
The biggest takeaway for me was using the (>) operator for output redirection. Usually, I just echo things to the terminal, but sending that data straight into a .txt file showed me how these scripts can be used to generate reports or config files on the fly. I also threw in a comment about aliases to show how I'd make this tool easier to launch in a real environment. This project really helped me see that Bash isn't just for fixing broken systems it's actually a pretty creative way to handle file management and user data.

```
(pranavi@oss-lab):~$ bash script5.sh
****************************************
      OSS CAPSTONE PROJECT SUMMARY
****************************************
Student Name : pranavi
Reg Number   : 24BCY10131
Project      : Open Source Audit (Git)
****************************************
Final Declaration:
I have completed the audit of Git software.
All 5 scripts (1-5) have been executed.
System environment verified as healthy.
Date: Tue Mar 31 18:50:05 IST 2026
****************************************
```

5. Project Timeline

This timeline is a guide, not a rigid rule. Some students will move faster on the technical parts; others will spend more time on the philosophical sections. Adjust based on your strengths — but do not leave the scripts to the final stretch.

Day	Focus	Tasks	Deliverable
1	**Setup**	Choose software. Research its origin story. Read the license text. Set up your Linux environment (VM or lab system). Install your chosen software.	*Choice confirmed*
2	**Philosophy**	Write Part A1 — the origin story. Take notes on what problem the software solved and for whom.	*Draft A1*
3	**License & Ethics**	Write Part A2 — license analysis. Write Part A3 — ethical reflection. These sections need your own thinking, not summaries.	*Draft A2 + A3*
4	**Linux Footprint**	Install software on Linux. Document directories, user, permissions. Write Part B.	*Draft Part B + screenshots*
5	**Scripts 1 & 2**	Write and test Script 1 (System Identity) and Script 2 (Package Inspector). Add them to your report with explanations.	*Scripts 1–2 working*
6	**Scripts 3 & 4**	Write and test Script 3 (Disk Auditor) and Script 4 (Log Analyzer). Test with a real log file.	*Scripts 3–4 working*
7	**Ecosystem**	Write Part C — FOSS ecosystem map. Research dependencies and community.	*Draft Part C*
8	**Comparison & Script 5**	Write Part D — comparison table and verdict. Write and test Script 5 (Manifesto Generator).	*Draft Part D + Script 5*
9	**Report polish**	Compile full report. Write introduction and conclusion. Add screenshots. Check page count (12–16 pages).	*Complete draft*
10	**Submission**	Final review. Submit report + 5 .sh files. Be ready to explain any script in a short viva if asked.	*Final submission*

6. Evaluation Rubric

Total marks: 100. The report carries 60 marks and the shell scripts carry 40 marks. There is no viva unless the evaluator suspects the work is not original — in which case any student may be called for a brief oral explanation.

Report (60 marks)

#	Component	Full marks	Passing (50%)	Notes
1	Origin story — depth, original research, storytelling	12	6	*Part A1*
2	License analysis — accuracy and understanding of freedoms	12	6	*Part A2*
3	Ethical reflection — quality of argument and original thinking	10	5	*Part A3*
4	Linux footprint — technical accuracy and screenshots	10	5	*Part B*
5	Ecosystem and LAMP connection	8	4	*Part C*
6	Open source vs proprietary comparison	8	4	*Part D*

Shell Scripts (40 marks — 8 marks each)

Each script is marked on three criteria:

Criterion	Marks	What we look for
Correctness — does it run without errors?	4	*Script executes on Linux and produces expected output*
Concepts — does it use the right shell constructs?	2	*Loops, if-then, case, variables used appropriately for the task*
Comments and documentation	2	*Every non-obvious line has a comment; script purpose is clear*

Academic integrity	All written sections must be in your own words. Code that is copy-pasted from the internet without understanding will be zero-marked. Running a script and explaining what it does in a conversation is considered a passing demonstration of understanding.

7. Reference Resources

Use these as starting points, not as sources to copy from. The report must be in your own words.

On open source philosophy

- GNU Project — 'The Free Software Definition': gnu.org/philosophy/free-sw.html
- Open Source Initiative — 'The Open Source Definition': opensource.org/osd
- Eric S. Raymond — 'The Cathedral and the Bazaar': catb.org/~esr/writings/cathedral-bazaar
- Linus Torvalds — 'Just for Fun' (autobiography, available at most libraries)

On Linux and shell scripting

- The Linux Command Line by William Shotts — freely available at linuxcommand.org
- GNU Bash Manual: gnu.org/software/bash/manual
- Linux man pages: man.cx (online) or type 'man <command>' in your terminal

For finding license texts

- SPDX License List: spdx.org/licenses
- Choose a License: choosealicense.com
- Your software's official repository on GitHub or GitLab — the LICENSE file is always there

A closing thought

Every tool you will use in your career — the editor, the compiler, the server, the database — was shaped by people who chose to build in the open and share their work freely. Understanding why they made that choice, and what it means for the world, is not a footnote to your technical education. It is the foundation of it.

8. Submission Requirements

All submissions are made through the VITyarthi portal. You must submit three things: a public GitHub repository link, a README.md inside the repository, and a project report PDF. All three are compulsory. Incomplete submissions will not be evaluated.

What to submit on the VITyarthi portal

Item	Details	Compulsory?
GitHub repo link	The repository must be public. It must contain all five .sh script files and a README.md. Name the repo: oss-audit-[rollnumber]	**Yes**
README.md	Must be inside the GitHub repository. Must include: student name and roll number, chosen software, description of each script, step-by-step instructions to run each script on Linux, and any dependencies required.	**Yes**

Project report PDF	Upload directly on the VITyarthi portal. Must follow the structure in Section 3. Must be 12 to 16 pages. Must be your own writing.	**Yes**

On the VITyarthi portal submission form, paste the GitHub repository URL, confirm the README is present in the repository, and upload the project report PDF. All three must be submitted together in a single submission — partial submissions will not be accepted.

Evaluation pipeline

Every submission goes through an automated evaluation pipeline before it reaches an evaluator. Be aware of the following checks that run on every submitted report and script set.

Check	What it detects	Consequence
Plagiarism detection	Report text copied from websites, Wikipedia, other students' submissions, or any published source without attribution. Cross-submission similarity between enrolled students is also flagged.	Zero on the full report. Referred to disciplinary committee if similarity exceeds 40%.
AI generation detection	Report sections generated by AI tools such as ChatGPT, Gemini, Claude, or similar. The pipeline checks for linguistic patterns, sentence uniformity, and structural signatures common to AI-generated text.	Flagged submissions are called for a mandatory viva. If the student cannot explain the content, zero marks are awarded.

A note on AI tools: you may use AI to understand concepts, debug scripts, or get explanations. You may not use AI to write your report. The distinction is the same as using a calculator versus asking someone else to sit your exam. The report must reflect your thinking, your words, and your conclusions.